Übungsblatt 4

JavaScript, Verteilte Architekturen, Accessibility, Barrierefreiheit

5430UE Web and Data Engineering

6. Mai 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Implementierung und dynamische Einbindung von Validierungslogik in HTML-Formulare mittels JavaScript
- Analyse von Herausforderungen verteilter Systeme
- Anwendung von Prinzipien des semantischen HTML
- Kennen grundlegende Problemfelder im Kontext Barrierefreiheit

HTML und Javascript: Luhn-Algorithmus zur Kreditkarten-Validierung

Übung 4.1: HTML und Javascript: Luhn-Algorithmus

Der Luhn-Algorithmus prüft, ob eine Kreditkartennummer formal korrekt ist (Tipp-Fehler-Erkennung).
Algorithmus:

1. Beginnend bei der vorletzten Ziffer wird jede zweite Ziffer (nach links gehend) mit 2 multipliziert
2. Ist das Ergebnis dieser Multiplikation größer als 9, bildet man die Quersumme (z.B. 16 ergibt 1 + 6 = 7).
   Alternativ kann man auch 9 vom Ergebnis abziehen (16 - 9 = 7)
3. Addiere nun alle Ziffern (die unveränderten und die neu berechneten)
4. Die Nummer ist gültig, wenn die Gesamtsumme ohne Rest durch 10 teilbar ist.

Beispiele: 4532015112830366 ist gültig, 1234567890123456 ist nicht gültig.

Übung 4.1: HTML und Javascript: Luhn-Algorithmus (Fortsetzung)

1. Implementieren Sie eine JavaScript-Funktion `isValidLuhn(number)`, die eine Kreditkartennummer als String entgegennimmt und `true` zurückgibt wenn die Nummer den Luhn-Check besteht, sonst `false`.

2. Implementieren Sie ein HTML-Dokument, das ein Eingabefeld für Nummern bietet. Bei jeder Eingabe soll die Funktion aus Teilaufgabe a) zur Prüfung aufgerufen werden. Nutzen Sie hierfür das oninput-Event, um auf jede Änderung des Feldinhalts (Eingabe, Löschen oder Einfügen) sofort zu reagieren. Je nach Auswertung der Prüfung soll unterhalb des Eingabefelds eine passende Ausgabe (*Gültig!*, *Ungültig!* oder *Fehlerhafte Eingabe!*) angezeigt werden.



Übung 4.1: Luhn-Algorithmus — Lösung

Zu 1) und 2):

Siehe Code.

Herausforderungen verteilter Systeme

Übung 4.2: Herausforderungen verteilter Systeme

Wir betrachten folgendes Szenario:
Ein Unternehmen migriert seinen Monolith auf 4 Server-Instanzen hinter einem Load Balancer.

Ordnen Sie folgende Vorfälle den Herausforderungen verteilter Systeme zu (*Netzwerk-Latenz* / *Partielle Ausfälle* / *Konsistenz* / *Koordination von Prozessen*):

Vorfall	Herausforderung
Server 2 fällt aus; 25 % der Requests schlagen fehl	?
Nutzer sieht nach dem Kauf kurz den alten Kontostand	?
Zwei Server verarbeiten gleichzeitig dieselbe Bestellung	?
API-Calls zwischen Services dauern 50 ms statt 0,1 ms	?

Übung 4.2: Herausforderungen verteilter Systeme — Lösung (1/2)

Vorfall	Herausforderung
Server 2 fällt aus; 25 % der Requests schlagen fehl	
Nutzer sieht nach dem Kauf kurz den alten Kontostand	
Zwei Server verarbeiten gleichzeitig dieselbe Bestellung	
API-Calls zwischen Services dauern 50 ms statt 0,1 ms	

Übung 4.2: Herausforderungen verteilter Systeme — Lösung (1/2)

Vorfall	Herausforderung
Server 2 fällt aus; 25 % der Requests schlagen fehl	**Partielle Ausfälle** — einzelne Knoten können unabhängig versagen
Nutzer sieht nach dem Kauf kurz den alten Kontostand	
Zwei Server verarbeiten gleichzeitig dieselbe Bestellung	
API-Calls zwischen Services dauern 50 ms statt 0,1 ms	

Übung 4.2: Herausforderungen verteilter Systeme — Lösung (1/2)

Vorfall	Herausforderung
Server 2 fällt aus; 25 % der Requests schlagen fehl	**Partielle Ausfälle** — einzelne Knoten können unabhängig versagen
Nutzer sieht nach dem Kauf kurz den alten Kontostand	**Konsistenz** — Replika noch nicht synchronisiert (eventual consistency)
Zwei Server verarbeiten gleichzeitig dieselbe Bestellung	
API-Calls zwischen Services dauern 50 ms statt 0,1 ms	



Übung 4.2: Herausforderungen verteilter Systeme — Lösung (1/2)

Vorfall	Herausforderung
Server 2 fällt aus; 25 % der Requests schlagen fehl	**Partielle Ausfälle** — einzelne Knoten können unabhängig versagen
Nutzer sieht nach dem Kauf kurz den alten Kontostand	**Konsistenz** — Replika noch nicht synchronisiert (eventual consistency)
Zwei Server verarbeiten gleichzeitig dieselbe Bestellung	**Koordination** — ohne Locking/Idempotenz bearbeiten mehrere Instanzen denselben Job
API-Calls zwischen Services dauern 50 ms statt 0,1 ms	

Übung 4.2: Herausforderungen verteilter Systeme — Lösung (1/2)

Vorfall	Herausforderung
Server 2 fällt aus; 25 % der Requests schlagen fehl	**Partielle Ausfälle** — einzelne Knoten können unabhängig versagen
Nutzer sieht nach dem Kauf kurz den alten Kontostand	**Konsistenz** — Replika noch nicht synchronisiert (eventual consistency)
Zwei Server verarbeiten gleichzeitig dieselbe Bestellung	**Koordination** — ohne Locking/Idempotenz bearbeiten mehrere Instanzen denselben Job
API-Calls zwischen Services dauern 50 ms statt 0,1 ms	**Netzwerk-Latenz** — Netzwerkaufrufe sind Größenordnungen langsamer als lokale Operationen

Übung 4.2: Herausforderungen verteilter Systeme — Lösung (2/2)

Zusatz - Begründungen:

1. Im Gegensatz zum Monolithen (Ganz-oder-gar-nicht) können in verteilten Systemen einzelne Komponenten unabhängig versagen. Die Software muss lernen, mit diesem „Teil-Defekt“ umzugehen und Anfragen stabil auf die verbleibenden Instanzen umzuleiten.

Übung 4.2: Herausforderungen verteilter Systeme — Lösung (2/2)

Zusatz - Begründungen:

1. Im Gegensatz zum Monolithen (Ganz-oder-gar-nicht) können in verteilten Systemen einzelne Komponenten unabhängig versagen. Die Software muss lernen, mit diesem „Teil-Defekt“ umzugehen und Anfragen stabil auf die verbleibenden Instanzen umzuleiten.
2. Datenänderungen benötigen Zeit, um auf alle Server-Instanzen repliziert zu werden. Greift ein Nutzer nach einem Schreibvorgang auf einen noch nicht aktualisierten Server zu, entstehen kurzzeitig widersprüchliche Datenstände.

Übung 4.2: Herausforderungen verteilter Systeme — Lösung (2/2)

Zusatz - Begründungen:

1. Im Gegensatz zum Monolithen (Ganz-oder-gar-nicht) können in verteilten Systemen einzelne Komponenten unabhängig versagen. Die Software muss lernen, mit diesem „Teil-Defekt“ umzugehen und Anfragen stabil auf die verbleibenden Instanzen umzuleiten.
2. Datenänderungen benötigen Zeit, um auf alle Server-Instanzen repliziert zu werden. Greift ein Nutzer nach einem Schreibvorgang auf einen noch nicht aktualisierten Server zu, entstehen kurzzeitig widersprüchliche Datenstände.
3. In einem Monolithen kann man einen Prozess einfach über den gemeinsamen Arbeitsspeicher sperren (Locking). Bei verteilten Instanzen wissen die Server ohne explizite Kommunikation nichts voneinander. Wenn der Load Balancer zwei schnell aufeinanderfolgende Klicks desselben Nutzers an zwei verschiedene Server schickt, greifen beide auf die Datenbank zu. Ohne eine zentrale Koordination (z. B. ein verteiltes Schloss/Distributed Lock oder Idempotenz-Token) führen beide Server die Transaktion aus.

Übung 4.2: Herausforderungen verteilter Systeme — Lösung (2/2)

Zusatz - Begründungen:

1. Im Gegensatz zum Monolithen (Ganz-oder-gar-nicht) können in verteilten Systemen einzelne Komponenten unabhängig versagen. Die Software muss lernen, mit diesem „Teil-Defekt“ umzugehen und Anfragen stabil auf die verbleibenden Instanzen umzuleiten.
2. Datenänderungen benötigen Zeit, um auf alle Server-Instanzen repliziert zu werden. Greift ein Nutzer nach einem Schreibvorgang auf einen noch nicht aktualisierten Server zu, entstehen kurzzeitig widersprüchliche Datenstände.
3. In einem Monolithen kann man einen Prozess einfach über den gemeinsamen Arbeitsspeicher sperren (Locking). Bei verteilten Instanzen wissen die Server ohne explizite Kommunikation nichts voneinander. Wenn der Load Balancer zwei schnell aufeinanderfolgende Klicks desselben Nutzers an zwei verschiedene Server schickt, greifen beide auf die Datenbank zu. Ohne eine zentrale Koordination (z. B. ein verteiltes Schloss/Distributed Lock oder Idempotenz-Token) führen beide Server die Transaktion aus.
4. Netzwerkaufrufe sind physikalisch bedingt um Größenordnungen langsamer als lokale Funktionsaufrufe im RAM. Diese Verzögerungen summieren sich bei vielen Service-Interaktionen schnell zu spürbaren Wartezeiten für den Endnutzer.

Web-Technologien kategorisieren

Übung 4.3: Web-Technologien kategorisieren

Ordnen Sie die folgenden Technologien/Standards in die richtige Kategorie ein:
`JSF, XML, HTTP, URI, HTML, CSS, SOAP, JSON, REST, Thymeleaf, JDBC, SQL`

Kategorie	Technologie
Protokoll	?
Adressierung	?
Markup-/Datenformat	?
Framework/Template-Engine	?
Datenbankzugriff	?
Architekturstil	?

Hinweis: Manche Technologien können mehreren Kategorien zugeordnet werden — wählen Sie die primäre Kategorie aus.

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	
Adressierung	
Markup-/Datenformat	
Framework/Template-Engine	
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP,
Adressierung	
Markup-/Datenformat	
Framework/Template-Engine	
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	
Markup-/Datenformat	
Framework/Template-Engine	
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	
Framework/Template-Engine	
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML,
Framework/Template-Engine	
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML, XML,
Framework/Template-Engine	
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML, XML, JSON,
Framework/Template-Engine	
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML, XML, JSON, CSS
Framework/Template-Engine	
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML, XML, JSON, CSS
Framework/Template-Engine	JSF (Java Server Faces),
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML, XML, JSON, CSS
Framework/Template-Engine	JSF (Java Server Faces), Thymeleaf
Datenbankzugriff	
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML, XML, JSON, CSS
Framework/Template-Engine	JSF (Java Server Faces), Thymeleaf
Datenbankzugriff	JDBC,
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML, XML, JSON, CSS
Framework/Template-Engine	JSF (Java Server Faces), Thymeleaf
Datenbankzugriff	JDBC, SQL
Architekturstil	

Übung 4.3: Web-Technologien kategorisieren — Lösung (1/3)

Kategorie	Technologie
Protokoll	HTTP, SOAP (baut auf HTTP auf),
Adressierung	URI
Markup-/Datenformat	HTML, XML, JSON, CSS
Framework/Template-Engine	JSF (Java Server Faces), Thymeleaf
Datenbankzugriff	JDBC, SQL
Architekturstil	REST

Übung 4.3: Web-Technologien kategorisieren — Lösung (2/3)

Anmerkung:

- **SOAP** ist technisch ein Protokoll (XML-basiertes Messaging), baut aber typischerweise auf HTTP auf.
- **REST** ist kein Protokoll, sondern ein Architekturstil.
- **XML** ist sowohl Datenformat als auch Basis für SOAP/WSDL-Beschreibungen.

Übung 4.3: Web-Technologien kategorisieren — Lösung (3/3)

Zusatz - Kurzerklärungen:

- **JSF**: Ein serverseitiges Java-Webframework zur Erstellung komponentenbasierter Benutzeroberflächen.
- **XML**: Eine Auszeichnungssprache zur strukturierten Darstellung und Speicherung von Daten.
- **HTTP**: Ein Protokoll zur Übertragung von Daten im Web zwischen Client und Server.
- **URI**: Ein Identifikator zur eindeutigen Benennung von Ressourcen im Internet.
- **HTML**: Die Standardsprache zur Strukturierung von Inhalten im Webbrowser.
- **CSS**: Eine Stylesheet-Sprache zur Gestaltung des Aussehens von HTML-Dokumenten.
- **SOAP**: Ein Protokoll zum Austausch strukturierter Informationen in Webservices, basierend auf XML.
- **JSON**: Ein leichtgewichtiges Datenformat zum Austausch von strukturierten Daten.
- **REST**: Ein Architekturstil für Webservices, der auf HTTP und zustandslosen Operationen basiert.
- **Thymeleaf**: Eine serverseitige Java-Template-Engine zur Generierung von HTML-Dokumenten.
- **JDBC**: Eine API zur Verbindung und Interaktion von Java-Anwendungen mit relationalen Datenbanken.
- **SQL**: Eine Sprache zur Definition, Abfrage und Manipulation von Daten in relationalen Datenbanken.



Semantisches HTML

Übung 4.4: Semantisches HTML (1/2)

Gegeben ist folgendes unsemantisches HTML:

```
<div class="header">
  <div class="nav">
    <div class="nav-item"><a href="/">Home</a></div>
    <div class="nav-item"><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="main">
  <div class="article">
    <div class="title">Produktname</div>
    <div class="text">Beschreibung...</div>
  </div>
</div>
<div class="footer">Copyright 2026</div>
```



Übung 4.4: Semantisches HTML (2/2)

1. Schreiben Sie die Struktur mit semantischen HTML5-Elementen um.

2. Warum ist semantisches HTML für Suchmaschinen und Screenreader wichtig?

Übung 4.4: Semantisches HTML — Lösung 1

1. Formulierung mit semantischen Elementen:

Unsemantisch (Vorher):

```
<div class="header">
  <div class="nav">
    <div class="nav-item"><a href="/">Home</a></div>
    <div class="nav-item"><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="main">
  <div class="article">
    <div class="title">Produktname</div>
    <div class="text">Beschreibung...</div>
  </div>
</div>
<div class="footer">Copyright 2026</div>
```

Semantisch (Nachher):

```
<header>
  <nav>
    <a href="/">Home</a>
    <a href="/products">Produkte</a>
  </nav>
</header>
<main>
  <article>
    <h1>Produktname</h1>
    <p>Beschreibung...</p>
  </article>
</main>
<footer>Copyright 2026</footer>
```

Übung 4.4: Semantisches HTML — Lösung 2

2. Warum semantisch?

Übung 4.4: Semantisches HTML — Lösung 2

2. Warum semantisch?

- **Suchmaschinen**: Crawler verstehen die Bedeutung der Inhalte — <h1> signalisiert Hauptüberschrift, <nav> Navigationsbereiche. Das verbessert SEO (Search Engine Optimization).

Übung 4.4: Semantisches HTML — Lösung 2

2. Warum semantisch?

- **Suchmaschinen**: Crawler verstehen die Bedeutung der Inhalte — <h1> signalisiert Hauptüberschrift, <nav> Navigationsbereiche. Das verbessert SEO (Search Engine Optimization).
- **Screenreader**: Blinde Nutzer navigieren per Tastatur durch Landmarks (`header`, `main`, `nav`). Ohne Semantik ist jedes gleichwertig — Navigation wird unmöglich.

Formulare und Accessibility

Übung 4.5: Formulare und Accessibility

1. Erstellen Sie ein barrierefreies HTML-Formular für eine Produktsuche mit:

   - Texteingabe für den Suchbegriff (Pflichtfeld)
   - Dropdown für Kategorie (Elektronik, Kleidung, Bücher)
   - Checkbox „Nur verfügbare Produkte”
   - Submit-Button

   Sie können als Grundlage die zur Verfügung gestellte Vorlage `formular_accessibility_vorlage.html` nutzen.

2. Welche drei HTML-Attribute sind für Barrierefreiheit unverzichtbar?

Übung 4.5: Formulare und Accessibility — Lösung 1)

1) HTML-Formular:

```
<form action="/search" method="GET">
  <label for="query">Suchbegriff</label>
  <input id="query" name="query" type="text" required placeholder="z.B. Laptop" />

  <label for="category">Kategorie</label>
  <select id="category" name="category">
    <option value="">Alle</option>
    <option value="electronics">Elektronik</option>
    <option value="clothing">Kleidung</option>
    <option value="books">Bücher</option>
  </select>

  <input type="checkbox" id="available" name="available" value="true"/>
  <label for="available">Nur verfügbare Produkte</label>

  <button type="submit">Suchen</button>
</form>
```

Übung 4.5: Formulare und Accessibility — Lösung 2)

2) Unverzichtbare Attribute:

1. `for / id` - **Verknüpfung**: Verbindet Label mit Input — Klick auf Label fokussiert das Feld.
2. `required`: Zeigt Pflichtfelder an und verhindert leere Submits (Browser- Validierung).
3. `type` (auf `<input>`): Bestimmt Eingabemodus und Tastaturlayout auf Mobilgeräten.

Barrierefreiheit

Übung 4.6: Barrierefreiheit

Barrierefreiheit ist nach EU Richtlinien ein wichtiger Aspekt in der Web-Entwicklung. Recherchieren und beantworten Sie folgende Fragen zum Thema Barrierefreiheit in Webanwendungen.

1. Barrieren können in Wahrnehmungs-, Verständnis- und Zugriffsprobleme klassifiziert werden. Ein typisches Wahrnehmungsproblem in Webapplikationen bilden fehlende Textalternativen für Medien. Nicht alle Videos haben beispielsweise Untertitel. Nennen und erklären Sie pro Problemkategorie 2 andere Beispiele. Welche Problemkategorie ist hauptsächlich technischer Natur?

2. Was verbirgt sich hinter dem Selbstbestimmungsprinzip, dem Zwei-Sinne Prinzip und dem Universelles Design Prinzip.

Übung 4.6: Barrierefreiheit — Lösung 1

Problemkategorien:

Übung 4.6: Barrierefreiheit — Lösung 1

Problemkategorien:

- **Wahrnehmungsprobleme**:
  - *Ungenügende Farbkontraste*: Rot-grün-kritische Farbkombinationen oder helle Schrift auf hellem Hintergrund erschweren das Lesen für Menschen mit Sehbeeinträchtigungen.
  - *Mangelnde Skalierbarkeit*: Texte oder Bilder lassen sich gar nicht vergrößern (feste Abmessungen) bzw. nicht ohne dass Inhalte abgeschnitten werden oder unleserlich werden.

Übung 4.6: Barrierefreiheit — Lösung 1

Problemkategorien:

- **Wahrnehmungsprobleme**:
  - *Ungenügende Farbkontraste*: Rot-grün-kritische Farbkombinationen oder helle Schrift auf hellem Hintergrund erschweren das Lesen für Menschen mit Sehbeeinträchtigungen.
  - *Mangelnde Skalierbarkeit*: Texte oder Bilder lassen sich gar nicht vergrößern (feste Abmessungen) bzw. nicht ohne dass Inhalte abgeschnitten werden oder unleserlich werden.
- **Verständnisprobleme**:
  - *Ungenügende Semantik*: Überschriften werden nur optisch hervorgehoben, aber nicht korrekt als HTML-Strukturelemente ausgezeichnet. Dadurch können assistive Technologien (z.B. Screenreader) die Seitenstruktur nicht richtig interpretieren.
  - *Komplexere Inhalte*: Unübersichtliche Navigation, Fachsprache oder lange verschachtelte Texte erschweren das Verständnis der Inhalte. Oft fehlt eine Formulierung in einfacher Sprache.

Übung 4.6: Barrierefreiheit — Lösung 1

- **Zugriffsprobleme**:
  - *Zu knappe Timeouts*. Eine Sitzung läuft ab, bevor ein Formular vollständig ausgefüllt werden kann.
  - *Eingabegeräteabhängigkeit*Eine Webanwendung ist nur per Maus bedienbar und unterstützt keine Tastaturnavigation.

  Die hauptsächlich technischen Problemkategorie sind die *Zugriffsprobleme*, da sie häufig durch technische Implementierungen wie fehlende Tastaturunterstützung oder inkompatible Schnittstellen entstehen.

Übung 4.6: Barrierefreiheit — Lösung 2

Prinzipien:

Übung 4.6: Barrierefreiheit — Lösung 2

Prinzipien:

- **Selbstbestimmungsprinzip**:
  Nutzende sollen die Möglichkeit haben, Darstellung und Bedienung einer Website an ihre individuellen Bedürfnisse anzupassen, z.B. durch veränderbare Schriftgrößen oder Kontraste.

Übung 4.6: Barrierefreiheit — Lösung 2

Prinzipien:

- **Selbstbestimmungsprinzip**:
  Nutzende sollen die Möglichkeit haben, Darstellung und Bedienung einer Website an ihre individuellen Bedürfnisse anzupassen, z.B. durch veränderbare Schriftgrößen oder Kontraste.
- **Zwei-Sinne Prinzip**:
  Informationen sollten über mindestens zwei unterschiedliche Wahrnehmungskanäle zugänglich sein, z.B. visuell durch Text und auditiv durch Screenreader-Ausgabe.

Übung 4.6: Barrierefreiheit — Lösung 2

Prinzipien:

- **Selbstbestimmungsprinzip**:
  Nutzende sollen die Möglichkeit haben, Darstellung und Bedienung einer Website an ihre individuellen Bedürfnisse anzupassen, z.B. durch veränderbare Schriftgrößen oder Kontraste.
- **Zwei-Sinne Prinzip**:
  Informationen sollten über mindestens zwei unterschiedliche Wahrnehmungskanäle zugänglich sein, z.B. visuell durch Text und auditiv durch Screenreader-Ausgabe.
- **Universelles Design Prinzip**:
  Digitale Angebote sollen von Anfang an so gestaltet werden, dass sie von möglichst allen Menschen ohne zusätzliche Speziallösungen genutzt werden können.

Materialien

Materialien zum Übungsblatt

- HTML-Vorlage zu Aufgabe *Formulare und Accessibility*: formular_accessibility_vorlage.html

Lösungen:

- Aufgabe *Luhn-Algorithmus*: Luhn.zip
- Aufgabe *Formulare und Accessibility*: : formular_accessibility_loesung.html